



tspredit Tutorials



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



Why tspredit?

Traditional frameworks treat preprocessing and modeling as separate steps. tspredit proposes the co-optimization of both, capturing interactions that fragmented approaches ignore.

The Problem

Current tools optimize model hyperparameters but treat preprocessing as a manual and isolated step.

The Gap

The interaction between preprocessing choices and model performance is ignored in existing frameworks.

The Central Question

Which combination of preprocessing, augmentation, and model produces the best forecasting accuracy?

Full Integration

Preprocessing + modeling co-optimized in a single reproducible pipeline.

Extensibility

Support for statistical models (ARIMA) and machine learning models (MLP) with a plugin architecture.

Connection with daltoolbox

Inherits the experiment infrastructure from DAL Toolbox to ensure reproducibility.

What you'll learn

10 progressive tutorials, from the simplest protocol to the complete pipeline with integrated tuning. Each tutorial answers a practical question and builds on the previous one.

tspredict Pipeline

01

Raw Data

Time series input

02

Preprocessing

Filtering and normalization

03

Augmentation

Training data expansion

04

Training

Predictive model fitting

05

Evaluation

Metrics and integrated optimization with `ts_integtune()`



Tip: if you already know the ARIMA protocols, you can skip directly to tutorial 04.

Access the tutorials at: <https://github.com/cefet-rj-dal/tspredict/tree/main/examples/tutorial>

ARIMA – One-Step Forecasting

1. Load and visualize

```
library(daltoolbox)
library(tspredit)
data(tsd)
plot_ts(x = tsd$x, y = tsd$y)
```

Load the package and inspect the series before any transformation.

2. Create the series object

```
ts <- ts_data(tsd$y, 0)
ts_head(ts, 5)
```

The argument `0` indicates no sliding window — ARIMA operates on the raw sequence.

3. Split train and test

```
samp <- ts_sample(ts, test_size = 1)
io_train <- ts_projection(samp$train)
io_test <- ts_projection(samp$test)
```

Reserve the last observation for out-of-sample evaluation. `ts_projection()` separates inputs and outputs.

4. Fit and predict

```
model <- ts_arima()
model <- fit(model,
  x = io_train$input,
  y = io_train$output)
prediction <- predict(model,
  x = io_test$input[1],
  steps_ahead = 1)
```

`ts_arima()` delegates order selection to `auto.arima()`. Use `attr(model, "params")` to view the chosen parameters.

ARIMA – Rolling-Origin Evaluation

Forecasting a single point does not reveal how the model behaves over time. Rolling-origin evaluation repeats one-step-ahead forecasts over a test segment, simulating real operational use.

Key difference in the code

```
samp <- ts_sample(ts, test_size = 5)
prediction <- predict(model,
  x = io_test$input,
  steps_ahead = 1)
```

Pass **the entire test segment** to `predict()` with `steps_ahead = 1`. Internally, the model forecasts one point, incorporates the observed value, and repeats — without refitting the model.

Why this matters

This protocol is closer to real-world use than forecasting a single point:

- Monitor tomorrow's value every day
- Update the forecast as new observations arrive
- Verify that short-term accuracy is stable over time

ARIMA – Multi-Step Ahead Forecasting

Instead of updating the model after each new observed value, we request several future values all at once. Forecast errors can accumulate over the horizon — which is why this protocol is more challenging.

Core Code

```
prediction <- predict(model,  
  x = io_test$input[1],  
  steps_ahead = 5)
```

Note: pass only `io_test$input[1]` — the first point of the test set. The model projects all 5 steps without observing the actual intermediate values.

Protocol Comparison

Protocol	steps_ahead	input in predict()
One step	1	io_test\$input[1]
Rolling-origin	1	io_test\$input (all)
Multi-step	5	io_test\$input[1]

The difference lies in *how many* steps to request and *which* input to provide.

Building the MLP Pipeline

Create Windows

```
sw_size <- 10
ts <- ts_data(tsd$y,
  sw_size)
```

Each row stores the 10 recent values used to predict the next one.

Split and Project

```
samp <- ts_sample(ts,
  test_size = 5)
io_train <- ts_projection(
  samp$train)
io_test <- ts_projection(
  samp$test)
```

Preserves temporal order in the split.

Configure and Fit

```
model <- ts_mlp(
  preprocess =
    ts_norm_gminmax(),
  input_size = 4,
  size = 4,
  decay = 0,
  maxit = 1000)
model <- fit(model,
  x = io_train$input,
  y = io_train$output)
```

`input_size=4`: uses the 4 most recent lags from each window.

Predict and Evaluate

```
prediction <- predict(model,
  x = io_test$input[1:1,],
  steps_ahead = 5)
ev_test <- evaluate(model,
  output, prediction)
```

Recursive 5-step forecast starting from the first test window.

Comparing Normalization Strategies

Normalization can be decisive for neural models. This tutorial isolates that decision — keeping the same MLP architecture — to show the effect of each strategy.

Reusable pattern: helper function

```
run_mlp <- function(preprocess,
                    label) {
  model <- ts_mlp(
    preprocess = preprocess,
    input_size = 4,
    size = 4,
    decay = 0,
    maxit = 1000)
  model <- fit(model,
    x = io_train$input,
    y = io_train$output)
  prediction <- as.vector(
    predict(model,
      x = io_test$input[1:1,],
      steps_ahead = 5))
  list(label = label,
    model = model,
    prediction = prediction,
    test_metrics = evaluate(model,
      as.vector(io_test$output),
      prediction)$metrics)
}
```

Run the comparison

```
res_gminmax <- run_mlp(
  ts_norm_gminmax(),
  "global min-max")

res_an <- run_mlp(
  ts_norm_an(),
  "adaptive normalization")
```

Compare metrics side by side

```
rbind(
  cbind(model = res_gminmax$label,
    res_gminmax$test_metrics),
  cbind(model = res_an$label,
    res_an$test_metrics)
)
```

MLP with Prior Filtering

Some time series contain short-term irregularities that hinder learning. A filter can be applied **before** building the sliding windows. The main change in this tutorial is the signal — not the model.

1. Apply the filter

```
filter_model <- ts_fil_smooth()
filter_model <- fit(filter_model,
  tsd$y)
y_filtered <- transform(
  filter_model, tsd$y)
```

Fit the filter on the full series and transform it before creating any windows.

2. Inspect the effect

```
plot_ts_pred(y = tsd$y,
  yadj = y_filtered)
```

Compare the original series with the filtered one before training any model.

3. Build pipeline on the filtered series

```
ts_filtered <- ts_data(
  y_filtered, 10)
samp <- ts_sample(ts_filtered,
  test_size = 5)
```

From here, the MLP pipeline follows exactly as in tutorial 04 — but on the filtered signal.

 **Practical lesson:** Filtering is not automatically beneficial. Treat it as an explicit modeling decision and compare pipelines with and without the filter using the same protocol.

MLP with Data Augmentation

Augmentation enriches the training set without touching the future test segment. The key point is not the specific technique, but the **pipeline logic**: augment only the training data.

Separate before augmenting

```
train_ts <- samp$train
test_ts  <- samp$test
io_test  <- ts_projection(test_ts)
```

Apply augmentation only on training data

```
augment_model <- ts_aug_jitter()
augment_model <- fit(
  augment_model, train_ts)
train_aug <- transform(
  augment_model, train_ts)
train_aug <- adjust_ts_data(
  train_aug)
io_train_aug <- ts_projection(
  train_aug)
```

Check the training set growth

```
data.frame(
  original = nrow(train_ts),
  augmented = nrow(train_aug)
)
# original: 27 → augmented: 54
```

The training set doubled in size. The test set remains intact.

Train with the augmented dataset

```
model <- ts_mlp(
  preprocess = ts_norm_gminmax(),
  input_size = 4,
  size = 4,
  decay = 0,
  maxit = 1000)
model <- fit(model,
  x = io_train_aug$input,
  y = io_train_aug$output)
```

The augmented dataset is used to train the model.

Complete Pipeline Code

Steps 1 & 2: Filter and Window

```
filter_model <- ts_fil_smooth()
filter_model <- fit(filter_model,
  tsd$y)
y_filtered <- transform(
  filter_model, tsd$y)

ts_filtered <- ts_data(
  y_filtered, 10)
samp <- ts_sample(ts_filtered,
  test_size = 5)
train_ts <- samp$train
test_ts <- samp$test
```

Steps 3 & 4: Augment and Fit

```
augment_model <- ts_aug_jitter()
augment_model <- fit(
  augment_model, train_ts)
train_aug <- transform(
  augment_model, train_ts)
train_aug <- adjust_ts_data(
  train_aug)
io_train <- ts_projection(train_aug)
io_test <- ts_projection(test_ts)

model <- ts_mlp(
  preprocess = ts_norm_an(),
  input_size = 4,
  size = 4,
  decay = 0,
  maxit = 1000)
model <- fit(model,
  x = io_train$input,
  y = io_train$output)
```

How to Use

Use `ts_norm_an()` for adaptive normalization.
The final forecast follows the same pattern as the previous tutorials.

Structure of the Comparison Code

The key is to prepare **two views of the same data** — raw sequence for ARIMA and sliding windows for MLP — and apply each protocol symmetrically.

Prepare the two views

```
ts_raw <- ts_data(tsd$y, 0)
ts_win <- ts_data(tsd$y, 10)
```

ARIMA uses `ts_raw`; MLP uses `ts_win`. Same data, different representations.

One-step vs. Rolling vs. Multi-step

```
# One step
predict(arima, x=io_raw_test$input[1],
        steps_ahead=1)
# Rolling-origin
predict(arima, x=io_raw_test$input,
        steps_ahead=1)
# Multi-step
predict(arima, x=io_raw_test$input[1],
        steps_ahead=5)
```

The difference between protocols lies in `steps_ahead` and which `x` is passed.

Compare metrics

```
rbind(
  cbind(model="ARIMA",
        evaluate(arima, obs,
                  pred_arima)$metrics),
  cbind(model="MLP",
        evaluate(mlp, obs,
                  pred_mlp)$metrics)
)
```

Use `rbind()` to place metrics side by side and facilitate visual comparison.

 **Core lesson:** The evaluation protocol must always match the intended use case. A model can be strong at one step ahead and weak at multiple steps.

How to Use ts_integtune()

Define the search space

```
tune <- ts_integtune(  
  input_size = 3:5,  
  base_model = ts_mlp(),  
  folds = 3,  
  preprocess = list(  
    ts_norm_gminmax(),  
    ts_norm_an()),  
  augment = list(  
    ts_aug_none(),  
    ts_aug_jitter()),  
  ranges = list(  
    size = 2:4,  
    decay = c(0, 0.01),  
    maxit = c(500)  
  )  
)
```

Define a **small and meaningful** set of alternatives — don't search everything that is possible.

Fit and inspect the result

```
model <- fit(tune,  
  x = io_train$input,  
  y = io_train$output)  
  
# View best configuration  
attr(model, "params")  
  
# View table of evaluated combinations  
head(attr(model,  
  "hyperparameters"))
```

The `params` object shows the winning configuration. The `hyperparameters` table records all evaluated combinations and their errors.

Predict with the tuned pipeline

```
prediction <- as.vector(  
  predict(model,  
    x = io_test$input[1:1,],  
    steps_ahead = 5))
```

After tuning, prediction follows exactly the same pattern as in the previous MLP tutorials.

❏ **Important:** If you want to include filtering, apply the filter **before** the tuning stage — `ts_integtune()` covers preprocessing, augmentation, input size, and model hyperparameters, but not upstream filtering.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

